Guía de estudio y ayuda memoria del código (`src`)

Documento vivo: **se actualiza al cerrar cada fase**. Sirve para buscar rápido qué hay en cada carpeta y en cada archivo, y para qué sirve.

Última actualización: 02/11/2026, al cierre de la Fase 5 (pesaje final y conciliación).

Convención de rutas: todo está bajo `src/main/java/ar/edu/iua/iw3/`, salvo que se diga otra cosa.

Columna **Origen**: *Cátedra* = ya venía del proyecto de clase · *F1* = creado o cambiado en la Fase 1 · *F2* = creado o cambiado en la Fase 2 · *F3* = creado o cambiado en la Fase 3 · *F4* = creado o cambiado en la Fase 4 · *F5* = creado o cambiado en la Fase 5.

---

1. El proyecto en un párrafo

Es un backend (API REST, sin pantallas) que administra **órdenes de carga de gas líquido**. Tres sistemas externos le hablan por HTTP:

- **SAP** manda la orden: camión, chofer, cliente, producto, fecha prevista y cuántos kg cargar (*preset*).
- **La balanza (TMS)** manda el peso del camión vacío (*tara*) y, al final, el peso final.
- **El sistema de carga** pide autorización con una contraseña y manda en tiempo real los datos del caudalímetro.

Con todo eso se calcula la **conciliación**: balanza contra caudalímetro.

Los estados de la orden son secuenciales, nunca vuelven atrás:

Estado	Significado
1	Pendiente de pesaje inicial
2	Con pesaje inicial registrado (se carga acá)
3	Cerrada para carga
4	Finalizada

2. Qué se hizo en cada fase

Fase	Contenido	Estado
0	Auditoría y línea base (no se tocó código)	Hecha
1	**Punto 1:** SAP envía la orden y se crea en estado 1	Hecha
2	**Punto 2:** la balanza envía la tara, se genera la contraseña de 5 dígitos y pasa a estado 2	Hecha
3	**Punto 3:** activación con contraseña y datos del caudalímetro en tiempo real	Hecha
4	**Punto 4:** cerrar la orden (estado 2 → 3) y bloquear nuevas lecturas	Hecha (PR #12)
5	**Punto 5:** pesaje final, estado 4 y conciliación (y los arreglos de la tara infinita y de la activación con bloqueo)	Hecha (rama `fase-5-conciliacion`)

3. Para qué sirve cada carpeta (lo que hay que recordar)

Cada carpeta responde **una pregunta**. Esa es la regla para saber dónde va un archivo nuevo.

Carpeta	Pregunta que responde	Qué NO debe tener
`model`	¿Qué datos hay y cómo se guardan?	Reglas, HTTP
`model/persistence`	¿Cómo se buscan y guardan?	Reglas, HTTP

Carpeta	Pregunta que responde	Qué NO debe tener
`business`	¿Qué reglas se aplican?	HTTP, SQL
`business/exception`	¿Qué errores de negocio existen?	—
`controllers`	¿Cómo entra y sale el pedido por HTTP?	Reglas
`integration`	¿Qué sistema externo llama?	Reglas
`security`	¿Qué se puede hacer sin token?	Lógica del dominio
`auth`	¿Quién es el usuario?	Lógica del dominio
`config`	¿Cómo se arma la aplicación?	Lógica del dominio
`util`	¿Qué herramientas se comparten?	Cosas de una sola capa

Camino de un pedido: `security` / `auth` (revisan el token) → `integration` o `controllers` (lo reciben) → `business` (decide) → `model/persistence` (guarda) → `model` (las tablas).

Regla para decidir dónde va algo nuevo: ¿qué cambiaría si esto cambiara? Si cambia la URL o el código de respuesta, `controllers`. Si cambia una regla, `business`. Si cambia una tabla, `model`.

---

4. `model/`: las fichas de datos

Cada ficha es una clase que se convierte en una tabla de la base de datos.

Archivo	Tabla	Qué es y para qué sirve	Origen
`Order.java`	`orders`	La orden de carga. Une camión, chofer, cliente y producto. Guarda número (único), fecha prevista, preset, estado, fecha de recepción, tara, fecha de pesaje inicial y contraseña de activación (con `@JsonIgnore`). Desde la Fase 3 también guarda, todos opcionales: fecha de activación, inicio y fin de carga y los últimos valores válidos de masa, densidad, temperatura y caudal. No tiene lista de detalles. Se llama `orders` porque `ORDER` es palabra reservada de SQL. Desde la Fase 5 guarda también el pesaje final (`finalWeight`) y su hora de recepción (`finalWeighingDateTime`), opcionales	F1 + F2 + F3 + F5
`OrderStatus.java`	—	Los 4 estados. En el JSON se publica el número (1 a 4)	F1
`OrderStatusConverter.java`	—	Traduce estado ↔ número para guardar 1 a 4 en la base, y no 0 a 3	F1
`OrderStatusLog.java`	`order_status_logs`	Historial: estado anterior (`null` al crear), estado nuevo, fecha, quién lo hizo y observaciones	F1
`Truck.java`	`trucks`	Camión: código externo (único), patente, descripción y lista de cisternas	F1
`TruckTank.java`	`truck_tanks`	Una cisterna: número y capacidad en litros. Una fila por cisterna	F1
`Driver.java`	`drivers`	Chofer: código externo, nombre, apellido y documento (opcional)	F1
`Customer.java`	`customers`	Cliente: código externo, razón social y contacto (opcional)	F1
`Product.java`	`gas_products`	Producto de gas: código externo, nombre y descripción (opcional). Usa `gas_products` porque `products` ya existe en bases viejas con otro esquema	F1 (reemplazó al del demo)

Archivo	Tabla	Qué es y para qué sirve	Origen
`OrderDetail.java`	`order_details`	Una lectura válida del caudalímetro (el histórico de la carga): masa acumulada, densidad, temperatura, caudal y estampa de tiempo (columna `reading_timestamp`). Apunta a su orden con `@ManyToOne` `LAZY` y `@JsonIgnore`	F3

Idea clave: camión, chofer, cliente y producto se identifican por su **código externo** (el que usa SAP), nunca por el `id` interno.

`model/persistence/`: los repositorios

Cada uno busca y guarda un tipo de ficha. Son interfaces: Spring arma el código.

Archivo	Qué hace	Origen
`OrderRepository.java`	Busca una orden por número y lista las órdenes paginadas, de la más reciente a la más antigua	F1
`OrderStatusLogRepository.java`	Trae el historial de una orden, del más antiguo al más reciente	F1
`TruckRepository.java`	Busca un camión por código externo	F1
`DriverRepository.java`	Busca un chofer por código externo	F1
`CustomerRepository.java`	Busca un cliente por código externo	F1
`ProductRepository.java`	Busca un producto por código externo	F1 (reemplazó al del demo)
`OrderDetailRepository.java`	Guarda y busca las lecturas del caudalímetro. `findByOrder(order, pageable)`: una página de lecturas por `id` ascendente. `averagesByOrder(order)`: promedios simples (`AVG`) de temperatura, densidad y caudal de la orden	F3 + F5
`ReadingAverages.java`	Proyección (interfaz) con los tres promedios que devuelve `averagesByOrder`; con cero lecturas los valores son `null`	F5

5. `business/`: las reglas

Archivo	Qué es	Origen
`IOrderBusiness.java`	**La lista** de operaciones sobre órdenes (el "menú"). Incluye `registerTare`, `activate`, `close`, `registerFinalWeight` y `reconciliation`	F1 + F2 + F3 + F4 + F5
`OrderBusiness.java`	**Las reglas de verdad** (la "cocina"). Implementa `add`, `registerTare`, `activate`, `close`, `registerFinalWeight` y `reconciliation`, con los privados `requireValidWeight` (peso de la tara y del pesaje final) y `buildReconciliation` (la única función que arma la conciliación)	F1 + F2 + F3 + F4 + F5
`Reconciliation.java`	Respuesta de la conciliación (9 campos: tara, pesaje final, producto cargado, neto por balanza, diferencia y tres promedios). Se calcula cada vez; no se guarda	F5
`OrderDetailValidator.java`	Función pura que evalúa las condiciones de descarte del caudalímetro (caudal, masa, masa anterior y densidad)	F3
`DetailResult.java`	DTO interno del resultado de una lectura: booleano `saved` y motivo `reason`	F3
`IOrderDetailBusiness.java`	Interfaz de negocio de las lecturas del caudalímetro (`add` y `list`)	F3
`OrderDetailBusiness.java`	Reglas de recepción de lecturas, descarte, guardado en `order_details` y actualización atómica de cabecera; `list` devuelve una página del histórico (404 si la orden no existe)	F3

Archivo	Qué es	Origen
`OrderValidator.java`	Revisa que la orden recibida esté completa y con valores lógicos	F1
`ActivationPasswordGenerator.java`	Genera contraseñas aleatorias de 5 dígitos (10000–99999) con `SecureRandom`	F2

`IOrderBusiness` frente a `OrderBusiness`

	`IOrderBusiness`	`OrderBusiness`
Qué tiene	Solo los nombres de las operaciones	Las operaciones con su código
Reglas	Ninguna	Todas

Los controllers piden `IOrderBusiness` y Spring les entrega `OrderBusiness`. Así los controllers no dependen de cómo está escrita la regla.

Operaciones de `IOrderBusiness` hoy: `list(Pageable)`, `load(orderNumber)`, `listStatusLog(orderNumber)`, `add(order, actor)`, `registerTare(orderNumber, weight, actor)`, `activate(orderNumber, password)`, `close(orderNumber, actor)`.

Qué hace `OrderBusiness.registerTare` (paso a paso)

1. Carga la orden con `load(orderNumber)` (si no existe, 404 `NotFoundException`).
2. Valida que el peso sea mayor que 0 con `!(weight > 0)` (si es 0, negativo o `NaN`, 400 `BadRequestException`).
3. Valida que la orden esté en estado 1 (`PENDING_INITIAL_WEIGHING`). Si no, 409 `ConflictException`.
4. Las comprobaciones 1 a 3 van **fuera del try/catch** para no convertirse en 500.
5. Dentro de `@Transactional(rollbackFor = Exception.class)`:
   - Asigna `tare` y `initialWeighingDateTime = LocalDateTime.now()`.
   - Genera la contraseña de 5 dígitos con `ActivationPasswordGenerator.generate()`.
   - Pasa la orden a estado 2 (`INITIAL_WEIGHING_REGISTERED`).
   - Persiste la orden con `orderDAO.save(order)`.
   - Asienta el cambio en el historial con `logStatus(order, 1, 2, actor, "Tara registrada: <peso> kg")`.
   - Ante cualquier falla no prevista, lanza `BusinessException` y la transacción se revierte.

Qué hace `OrderBusiness.activate` (paso a paso) — F3

1. Si falta `orderNumber` o `password`, 400 `BadRequestException`.
2. Carga la orden con `load(orderNumber)` (si no existe, 404).
3. Si la orden no está en estado 2, 409 `ConflictException` (informa el estado actual).
4. Si la contraseña no coincide con `activationPassword`, 403 `ForbiddenException`. El mensaje **no repite ninguna contraseña**.
5. Las comprobaciones 1 a 4 van **fuera del try/catch**, como en `registerTare`.
6. Si `activationDateTime` es nulo, lo guarda con la hora actual (`saveAndFlush`, para que un error de base salga dentro del `try`). Si ya estaba, **no lo toca**: la activación es idempotente (activar de nuevo con la contraseña correcta devuelve el mismo preset).
7. No cambia el estado (la orden sigue en 2) y **no escribe en el historial** de estados. Devuelve la orden; el controller responde el `preset`.

Qué hace `OrderBusiness.close` (paso a paso) — F4

1. Busca la orden con `loadForUpdate` → `findByOrderNumberForUpdate` (`SELECT … FOR UPDATE`): **bloquea la fila** hasta el fin de la transacción, así un cierre y una lectura de la misma orden se procesan de a uno. Si no existe, 404.
2. Si la orden no está en estado 2, 409 `ConflictException` ("no se puede cerrar … (estado X)"). Cerrar dos veces también da 409: **no** es idempotente.
3. Las comprobaciones 1 y 2 van **fuera del try/catch**.
4. Dentro de `@Transactional(rollbackFor = Exception.class)`: recuerda el estado anterior (2), pasa a `CLOSED_FOR_LOADING` (3), guarda con `saveAndFlush` y escribe el historial con `logStatus` (de 2 a 3, usuario del token, "Orden cerrada para carga").
5. **No toca** las fechas de carga, los últimos valores ni la contraseña. La fecha del cierre queda en `order_status_logs`.
6. El bloqueo de lecturas no es código nuevo: `OrderDetailBusiness.add` ya rechaza con 409 todo lo que no esté en estado 2.

Qué hace `OrderBusiness.registerFinalWeight` (paso a paso) — F5

1. Busca la orden **con bloqueo** (`loadForUpdate`); si no existe, 404.
2. `requireValidWeight`: peso ≤ 0 o `NaN` → 400 ("weight debe ser mayor que 0 (kg)"); infinito → 400 ("weight debe ser un número finito").
3. Si la orden no está en estado 3, 409. Si no tiene tara, 409.
4. Si el peso es menor que la tara, 400 (un peso **igual** a la tara es válido: una orden cerrada sin lecturas no cargó nada).
5. Las comprobaciones 1 a 4 van **fuera del try/catch**, en una transacción con `rollbackFor = Exception.class`.
6. Dentro del `try`: guarda `finalWeight` y `finalWeighingDateTime` (hora del servidor), pasa la orden a estado 4 (`FINISHED`) con `saveAndFlush` y agrega **una** fila 3 → 4 al historial ("Pesaje final registrado", usuario del token).
7. Devuelve `buildReconciliation(order)`: producto cargado = `lastAccumulatedMass` (0.0 si no hubo lecturas); neto por balanza = pesaje final − tara; diferencia = neto − producto cargado; promedios = `AVG` de `order_details` (`null` sin lecturas). Sin redondeo y sin guardar nada.

`reconciliation(orderNumber)` (el GET) busca la orden sin bloqueo, exige el estado 4 (409 si no) y usa la **misma** `buildReconciliation`.

Qué hace `OrderDetailBusiness.add` (paso a paso) — F3

1. Carga la orden con bloqueo de fila (`loadForUpdate` → `findByOrderNumberForUpdate`; 404 si no existe).
2. Valida presencia obligatoria de los cuatro valores numéricos (`accumulatedMass`, `density`, `temperature`, `flowRate`). Si alguno es nulo, 400 `BadRequestException`.
3. Valida que la orden esté en estado 2 (`INITIAL_WEIGHING_REGISTERED`) y activada (`activationDateTime != null`). Si no, 409 `ConflictException`.
4. Evalúa descarte con `OrderDetailValidator.discardReasons(...)` pasando la última masa de la cabecera. Si hay motivos, devuelve `DetailResult.discarded(...)` sin persistir nada.
5. Las comprobaciones 1 a 4 van **fuera del try/catch**.
6. Dentro de `@Transactional(rollbackFor = Exception.class)`:
   - Estampa = `LocalDateTime.now()`: la hora de recepción del servidor (el cliente no la manda).
   - Guarda `OrderDetail` en `order_details`.
   - Actualiza la cabecera: `lastAccumulatedMass`, `lastDensity`, `lastTemperature`, `lastFlowRate`, `loadingStartDateTime` (solo la primera vez) y `loadingEndDateTime` (siempre).
   - Confirma con `orderDAO.saveAndFlush(order)` para capturar errores de base dentro del `try`.

- Devuelve `DetailResult.saved()`.

7. Si ocurre una excepción técnica dentro del try, la captura, loguea y lanza `BusinessException` (500 con `StandartResponse`).

Qué hace `OrderBusiness.add` (paso a paso)

1. Valida con `OrderValidator`. Si hay errores, los devuelve **todos juntos** (400).
2. Revisa que el número de orden no exista (si existe, 302).
3. Sincroniza camión, chofer, cliente y producto por código externo: si existen los actualiza, si no los crea. Las cisternas solo se reemplazan si cambiaron.
4. Crea la orden en **estado 1** y le pone la fecha de recepción. El cliente no puede fijar ni el estado ni el `id`: se copian solo los campos permitidos.
5. Anota el historial: de nada a estado 1, con el usuario del token.
6. Todo ocurre **en una transacción**: si algo falla, no queda ninguna fila a medias.

Qué revisa `OrderValidator`

- `orderNumber` entero mayor que 0; fecha prevista presente; `preset` mayor que 0.
- Camión: código y patente; al menos una cisterna; cada cisterna con número y litros mayores que 0; sin números repetidos.
- Chofer: código, nombre y apellido.
- Cliente: código y razón social.
- Producto: código y nombre.
- Los textos no pueden pasar el largo de su columna.

`business/exception/`: errores de negocio

Archivo	Se convierte en	Origen
`NotFoundException.java`	404 (no se encontró)	Cátedra
`FoundException.java`	**302** (ya existe; convención de la cátedra para duplicados)	Cátedra
`ConflictException.java`	**409** (conflicto con el estado actual de la orden)	**F2**
`ForbiddenException.java`	**403** (operación no permitida aunque haya token: contraseña de activación incorrecta)	**F3**
`BusinessException.java`	500 (falla interna)	Cátedra
`BadRequestException.java`	400 (datos inválidos)	**F1**

---

6. `controllers/`: la capa HTTP

No tienen reglas: reciben el pedido, llaman al negocio y devuelven el código.

Archivo	Qué hace	Origen
`OrderRestController.java`	Consultas: lista paginada, una orden por número, historial de estados, lecturas guardadas paginadas (`/{n}/details`) y conciliación (`/{n}/reconciliation`, solo estado 4)	F1 + F3 + F5
`BaseRestController.java`	Clase madre de los controllers. Tiene `getUserLogged()`: el usuario del token, de donde sale el "quién" del historial	Cátedra + F1
`Constants.java`	Todas las URLs en un solo lugar. Incluye `URL_INTEGRATION_TMS` y `URL_INTEGRATION_LOADER`	Cátedra + F1 + F2 + F3

7. `integration/`: un subpaquete por sistema externo

Hoy SAP, TMS (balanza) y loader (sistema de carga).

Archivo	Qué hace	Origen
`sap/controllers/OrderSapRestController.java`	La puerta de SAP. Recibe la orden, la pasa al negocio y responde 201 (con header `location`), 302, 400 o 500	F1
`tms/model/TareRequest.java`	DTO del pesaje inicial con `double weight` (kg)	F2
`tms/model/TareResponse.java`	DTO de respuesta de la balanza con `Integer activationPassword` (5 dígitos)	F2
`tms/model/FinalWeightRequest.java`	DTO del pesaje final con `double weight` (kg); sin fecha: la estampa la pone el servidor	F5
`tms/controllers/OrderTmsRestController.java`	La puerta de la balanza (TMS). Recibe la tara en `POST /integration/tms/orders/{n}/tare`, llama a `registerTare` y devuelve la contraseña (200, 400, 404, 409 o 500). También recibe el pesaje final en `POST /integration/tms/orders/{n}/final-weight`, llama a `registerFinalWeight` y devuelve la conciliación (200, 400, 404, 409 o 500)	F2 + F5
`loader/model/ActivationRequest.java`	DTO del pedido de activación: `Integer orderNumber` y `Integer password` (`Integer` para distinguir un campo ausente, 400, de uno informado)	F3
`loader/model/ActivationResponse.java`	DTO de respuesta de la activación con `Double preset` (kg a cargar)	F3
`loader/model/DetailRequest.java`	DTO del pedido de lecturas: `Double` para los 4 campos numéricos (sin `timestamp`: la estampa la pone el servidor)	F3
`loader/model/DetailResponse.java`	DTO de respuesta a la recepción de lecturas con `boolean saved` y `String reason`	F3
`loader/controllers/OrderLoaderRestController.java`	La puerta del sistema de carga. Expone `POST /activation` (Preset), `POST /orders/{orderNumber}/details` (lecturas en tiempo real con descarte y actualización de cabecera) y `POST /orders/{orderNumber}/close` (cierre, estado 2 → 3)	F3 + F4

8. `security/` y `auth/`

`security` define qué es público. `auth` maneja usuarios, login y token. Casi todo viene de la cátedra.

Archivo	Qué hace	Origen
`security/SecurityConfiguration.java`	Rutas públicas y reglas. Todo exige token salvo login, `/error`, Swagger y `/demo/**`. Define la encriptación BCrypt y la sesión sin estado. Se le permitió `/error` para que los errores de formato no salgan como 401 mudos	Cátedra + F1
`auth/User.java`	Usuario (`users`). Valida cuenta vencida, bloqueada o deshabilitada. Tiene sus roles	Cátedra
`auth/Role.java`	Rol (`roles`)	Cátedra
`auth/UserRepository.java`	Busca un usuario por nombre o email	Cátedra
`auth/RoleRepository.java`	Acceso a la tabla de roles	Cátedra
`auth/IUserBusiness.java` y `UserBusiness.java`	Cargar usuario, cambiar contraseña, habilitar/deshabilitar, listar	Cátedra
`auth/BadPasswordException.java`	Error de contraseña incorrecta	Cátedra

Archivo	Qué hace	Origen
`auth/custom/CustomAuthenticationManager.java`	Valida usuario, estado de cuenta y contraseña (BCrypt)	Cátedra
`auth/controller/AuthRestController.java`	`POST /api/v1/login` : emite el token. También tiene `GET /demo/encodepass` (devuelve el hash BCrypt de una contraseña; sirve para crear usuarios de prueba). Se le quitó el `produces=text/plain` para que el login fallido devuelva su mensaje	Cátedra + F1
`auth/filters/AuthConstants.java`	Constantes del token: clave, vencimiento (1 h), nombre del header y del parámetro	Cátedra
`auth/filters/JWTAuthorizationFilter.java`	En cada pedido lee el token (header `Bearer` o `?authtoken=`) y deja al usuario identificado	Cátedra

Token (JWT): se firma, no se cifra. Vence a la hora. Dura aunque después deshabiliten al usuario, porque no consulta la base.

9. `config/` y `util/`

Archivo	Qué hace	Origen
`config/OpenApiConfig.java`	Configura Swagger: título y botón *Authorize* para pegar el token	F1
`util/StandartResponse.java`	El JSON de error: `message` , `code` , `devInfo` . El nombre con errata viene de la cátedra	Cátedra
`util/IStandartResponseBusiness.java` y `StandartResponseBusiness.java`	`build(status, ex, message)` arma ese JSON	Cátedra

10. Raíz de `src/main/java` y otras carpetas de `src`

Archivo	Qué hace	Origen
`Iw3Application.java`	Arranque de la app. Fija la zona horaria y loguea el perfil activo. Se le sacaron componentes del demo	Cátedra + F1
`src/main/resources/application.properties`	Conexión a la base y configuración general. Incluye la zona horaria de Argentina para la base (`hibernate.jdbc.time_zone`)	Cátedra + F1
`src/main/resources/application-mysqldev.properties` y `-mysqlprod.properties`	Perfiles de desarrollo y producción	Cátedra
`src/test/java/.../Iw3ApplicationTests.java`	Un test: que la aplicación arranque (`contextLoads`). Necesita MySQL	Cátedra
`src/test/java/.../ActivationPasswordGeneratorTest.java`	Prueba unitaria del generador de contraseñas (rango 10000–99999 y entropía). Sin base de datos	F2
`src/test/java/.../OrderDetailValidatorTest.java`	Pruebas unitarias de las reglas de descarte del caudalímetro (válidas, masa igual, caudal, masa, densidad, NaN, motivos acumulados). Sin Spring Boot	F3
`src/test/java/.../OrderBusinessCloseTest.java`	Pruebas unitarias del cierre con Mockito (repositorios simulados): 404, 409 en los estados 1, 3 y 4, transición 2 → 3 con una fila de historial, actor `"sistema"` por defecto y cabecera sin cambios. Sin base de datos	F4
`src/test/java/.../OrderBusinessTareTest.java`	Pruebas de la validación del peso de la tara con Mockito: infinito → 400, el mensaje de ≤ 0 y NaN no cambia y una tara válida pasa al estado 2 (H-26)	F5
`src/test/java/.../OrderBusinessActivateTest.java`	Pruebas de la activación con Mockito: 400, 404, 409, 403 sin contraseñas en el mensaje, primera activación e idempotencia; verifica que busca con bloqueo (H-25)	F5

Archivo	Qué hace	Origen
`src/test/java/.../OrderBusinessFinalWeightTest.java`	Pruebas del pesaje final y la conciliación con Mockito: 404, pesos inválidos, 409 por estado y por falta de tara, peso menor o igual a la tara, una sola fila 3 → 4, el ejemplo numérico, orden sin lecturas y la consulta en estado 4	F5

11. Endpoints que existen hoy

Método y ruta	Qué hace	Respuestas
`POST /api/v1/login`	Usuario y contraseña (formulario); devuelve el token en texto	200, 401
`POST /api/v1/integration/sap/orders`	**Punto 1:** SAP envía la orden	201 + `location`, 302, 400, 401, 500
`POST /api/v1/integration/tms/orders/{n}/tare`	**Punto 2:** la balanza envía la tara; devuelve la contraseña de activación de 5 dígitos	200, 400, 401, 404, 409, 500
`POST /api/v1/integration/loader/activation`	**Punto 3:** el sistema de carga se activa con `orderNumber` y `password`; devuelve el `preset`. Idempotente; la orden sigue en estado 2	200, 400, 401, 403, 404, 409, 500
`POST /api/v1/integration/loader/orders/{n}/details`	**Punto 3:** una lectura del caudalímetro. Responde `{ saved, reason }`: guardada o descartada, siempre 200	200, 400, 401, 404, 409, 500
`POST /api/v1/integration/loader/orders/{n}/close`	**Punto 4:** cierra la orden (estado 2 → 3) y desde ahí se rechazan las lecturas. Sin cuerpo; registra el cambio en el historial con el usuario del token. Responde la orden	200, 401, 404, 409, 500
`POST /api/v1/integration/tms/orders/{n}/final-weight`	**Punto 5:** la balanza envía el pesaje final `{ weight }`; la orden pasa de 3 a 4, se registra el cambio y responde la conciliación. No es idempotente; peso igual a la tara válido, menor 400	200, 400, 401, 404, 409, 500
`GET /api/v1/orders/{n}/reconciliation`	**Punto 5:** la conciliación (tara, pesaje final, producto cargado, neto, diferencia y promedios), solo en estado 4. Se calcula, no se guarda	200, 401, 404, 409, 500
`GET /api/v1/orders?page=&size=`	Lista paginada (`size` 20 por defecto, máximo 100). Responde `content` y `page`	200
`GET /api/v1/orders/{n}`	Una orden (no muestra `activationPassword`)	200, 404
`GET /api/v1/orders/{n}/status-log`	Historial de estados	200, 404
`GET /api/v1/orders/{n}/details?page=&size=`	Lecturas guardadas de la orden, paginadas por `id` ascendente (`size` 20 por defecto, máximo 100)	200, 404
`/swagger-ui.html`, `/v3/api-docs`	Documentación	públicos
`GET /demo/encodepass?password=`	Hash BCrypt (demo de la cátedra, público; cerrar antes de desplegar)	200

Códigos: **200** éxito (en las lecturas, también una lectura descartada: `saved: false`) · **201** creada · **302** ya existe (convención de la cátedra) · **400** datos inválidos, campo ausente, peso ≤ 0 o infinito, o pesaje final menor que la tara ·

401 sin token o inválido · **403** contraseña de activación incorrecta · **404** no existe · **409** la orden no está en el estado que pide la operación (o no está activada, en las lecturas; o no está en estado 4, en la conciliación) · **500** falla interna.

12. Fuera de `src`

Archivo	Para qué sirve
`pom.xml`	Dependencias y build (Maven)
`docker-compose.yml`	La base MySQL (contenedor `iw3-gas-mysql`, puerto 3306, base `iw3_db`)
`CLAUDE.md`	Estado del proyecto y reglas del equipo
`docs/plan-de-trabajo.md`	Plan completo y decisiones
`docs/fase-0-auditoria.md`, `docs/fase-1-recepcion-ordenes.md`, `docs/fase-2-pesaje-inicial.md`, `docs/fase-3-carga-tiempo-real.md`, `docs/fase-4-cierre-orden.md`, `docs/fase-5-conciliacion.md`	Informe de cada fase y guía de estudio con preguntas del profesor
`docs/fase-2-division-de-trabajo.md`	División del trabajo entre los 4 desarrolladores para la Fase 2
`docs/fase-3-division-de-trabajo.md`	División del trabajo para la Fase 3 (3 partes, trabajo secuencial), con los prompts y el registro de cada parte
`docs/fase-5-division-de-trabajo.md`	División del trabajo para la Fase 5 (2 partes, trabajo secuencial: Calvo y Alasino), con las decisiones D1 a D14, los prompts y el registro de cada parte (F5)
`docs/fase-5-conciliacion.md`	Informe de la Fase 5, **auditoría final del segundo parcial** (matriz de trazabilidad contra el PDF) y guía de estudio (F5)
`docs/postman/`	Colección Postman: 151 pedidos y 358 verificaciones en las carpetas 00 a 06. La 05 prueba el pesaje final y la conciliación; la 06 simula el **circuito completo 1 → 4** con *Tests* encadenados y su propio login (F5)
`docs/sql/seed-dev-users.sql`	Usuario de prueba `admin` (solo desarrollo)
`docs/pdf/`	PDF de los informes y de esta guía

13. Qué se borró en la Fase 1

30 archivos del demo de la cátedra: `integration/cli1`, `integration/cli2`, los `config/profile/*ScanConfig`, `util/JsonUtiles`, productos y categorías de ejemplo, y sus archivos de configuración. Siguen en el historial de git.

14. Decisiones que conviene saber defender

Decisión	Por qué
El `@RequestBody` es la entidad y el negocio copia solo los campos permitidos (SAP)	Siguiendo a la cátedra, pero evitando que el cliente fije el estado o el `id`
DTOs dedicados `TareRequest` y `TareResponse` (TMS)	La balanza solo manda el peso y recibe una clave; desacopla el contrato externo de la entidad
Transacción con `rollbackFor = Exception.class`	Las excepciones de negocio son *checked* y Spring no revierte por ellas
Duplicado = 302	Convención de la cátedra. 409 sería más preciso
Estado incorrecto = 409 Conflict	La orden existe y el JSON es correcto, pero el estado actual choca con la operación
Datos inválidos = 400	Extensión nuestra: no es culpa del servidor

Decisión	Por qué
Contraseña oculta con `@JsonIgnore`	Secreto compartido solo entre balanza y sistema de carga: no se expone en consultas
`SecureRandom` para la contraseña de 5 dígitos	Evita previsibilidad de contraseñas de carga de combustible
Comprobaciones 404/400/409 fuera del `try/catch`	Evita que el `catch (Exception)` las transforme en HTTP 500 genérico
Estado guardado como 1 a 4	La consigna habla de estados 1 a 4
Columnas de tara nulables (`Double`, `Integer`)	Permite coexistir con órdenes previas en `ddl-auto=update` sin romper la base
Tablas `orders` y `gas_products`	`ORDER` es reservada; `products` ya existe en bases viejas
Validación manual en el negocio	La cátedra no usa Bean Validation
`hibernate.jdbc.time_zone`	Sin eso la base guardaba la hora 3 h adelantada
Contraseña incorrecta = 403 (F3)	Hay token válido pero la operación no está permitida; un 401 se confundiría con un problema del token
Activación idempotente (F3)	Activar de nuevo devuelve el mismo preset y no cambia `activationDateTime`
Lectura descartada = 200 con `saved: false` (F3)	Descartar no es un error: el caudalímetro sigue mandando datos
Pedidos con `Double` e `Integer` (F3)	Un campo ausente llega como `null` y da 400; con primitivos llegaría como 0 y se descartaría
Masa anterior = la de la cabecera (F3)	No se recorre el histórico en cada lectura; una masa igual a la anterior es válida
`saveAndFlush` en las lecturas y la activación (F3)	Un error de base sale dentro del `try` y responde con `StandartResponse`
Cierre solo desde el estado 2, no idempotente (F4)	Los estados avanzan de a uno; cerrar dos veces es un error del cliente y no debe sumar otra fila al historial
Cierre con bloqueo pesimista (F4)	Hibernate escribe todas las columnas: sin bloqueo, un cierre podría pisar la cabecera que acaba de actualizar una lectura
Sin campo `closeDateTime` (F4)	La fecha y el usuario del cierre ya quedan en `order_status_logs`
Cierre sin exigir activación ni lecturas (F4, decisión 30, a confirmar por el equipo)	La consigna solo pide el estado esperado

15. Buscador rápido: "si quiero..., voy a..."

Si quiero...	Voy a
Ver qué datos lleva la orden	`model/Order.java`
Cambiar qué se le exige a SAP	`business/OrderValidator.java`
Cambiar lo que pasa al recibir la orden	`business/OrderBusiness.java` (`add`)
Cambiar la regla de pesaje inicial / tara	`business/OrderBusiness.java` (`registerTare`)
Cambiar la regla de activación de la carga	`business/OrderBusiness.java` (`activate`)
Cambiar la generación de contraseñas	`business/ActivationPasswordGenerator.java`
Cambiar las reglas de descarte de las lecturas	`business/OrderDetailValidator.java`
Cambiar lo que pasa al recibir una lectura	`business/OrderDetailBusiness.java` (`add`)

Si quiero...	Voy a
Cambiar la regla del cierre de la orden	`business/OrderBusiness.java` (`close`)
Cambiar un código HTTP	El controller del endpoint (`integration/sap/...`, `integration/tms/...`, `integration/loader/...` o `controllers/...`)
Agregar una URL	`controllers/Constants.java` y el controller
Abrir o cerrar una ruta sin token	`security/SecurityConfiguration.java`
Ver cómo se arma el token	`auth/controller/AuthRestController.java`
Cambiar el formato de error	`util/StandartResponseBusiness.java`
Ver los estados de la orden	`model/OrderStatus.java`
Cambiar la conexión a la base	`src/main/resources/application.properties`

16. Cómo mantener este documento

Al cerrar cada fase:

1. Agregar a las tablas los archivos nuevos y marcar los cambiados con la fase (F2, F3...).
2. Actualizar la tabla de endpoints (sección 11) y de fases (sección 2).
3. Actualizar la fecha de la cabecera.
4. Regenerar el PDF:

```
node docs/pdf/md2pdf.js docs/guia-estudio-src.md docs/pdf/guia-estudio-src.pdf "Guía de estudio del código"
```